

Dart Console Library System

Assignment 1

Student Name	Fauzan Baig
Course	B.Tech Computer Science Engineering
Language	Dart
Assignment	Dart Basics Scripts
Roll No	150096724092
Cohort	Elon Musk

The objective of this assignment was to create a Dart console program that uses variables, loops, functions, and Object-Oriented Programming (OOP), including classes and inheritance, to model a simple library system. The program represents books and library members and provides operations such as adding, displaying, searching, borrowing, and returning books.

2. Objectives

- Understand classes and objects in Dart.
- Use constructors to initialize object data.
- Apply inheritance using a specialized library member.
- Use List, Map, and Set collections.
- Use functions and loops to process library data.
- Build and run a simple console application.

3. Program Structure

- LibraryBook – stores book ID, title, author, and borrowing status.
- LibraryMember – stores member ID and name.
- PremiumLibraryMember – extends LibraryMember and demonstrates inheritance.
- Library – manages books, members, borrowed-book information, and library operations.

4. Concepts Used

Classes and Objects

Classes act as blueprints for books, members, and the library. Objects represent actual books and members.

Constructors

Constructors initialize values such as IDs, titles, authors, and names when objects are created.

Inheritance

PremiumLibraryMember extends LibraryMember, reusing the parent's data and behavior while adding its own behavior.

Collections

Lists store books and members, a Map tracks borrowed books against members, and a Set stores unique member names.

Functions and Loops

Functions perform library operations, while loops process books and members.

5. Working of the Library System

1. Create a Library object.
2. Add books.
3. Add members.
4. Display books and members.
5. Search for a book by title.
6. Borrow a book using book and member IDs.
7. Update the book status and borrowed-book Map.
8. Return the book and update its status.

6. Screenshots

```
fauzan:lib$ dart main.dart
===== TASK 1: CLASS =====
Book: The Alchemist
Author: Paulo Coelho

===== TASK 2: CONSTRUCTOR =====
Name: Fauzan Baig
Age: 19

===== TASK 3: METHODS =====
Addition: 15
Subtraction: 5
Multiplication: 50

===== TASK 4: INHERITANCE =====
Member: Aman
Aman is a Premium Member.

===== TASK 5: MIXIN =====
Library Report
This message comes from a mixin.

===== TASK 6: INTERFACE =====
Book title: Atomic Habits

===== TASK 7: LIST =====
Books in the list:
The Alchemist
Atomic Habits
Harry Potter
After adding a book:
[The Alchemist, Atomic Habits, Harry Potter, The Hobbit]
```

```

===== TASK 8: MAP =====
Student Map:
1 : Rahul
2 : Aman
3 : Riya
After adding a student:
{1: Rahul, 2: Aman, 3: Riya, 4: Neha}

===== TASK 9: SET =====
Subjects:
{Dart, Flutter, Java}
After adding Python:
{Dart, Flutter, Java, Python}

===== TASK 10: LIBRARY SYSTEM =====
Book added successfully.
Book added successfully.
Book added successfully.
Member added successfully.
Member added successfully.

```

```

===== LIBRARY SYSTEM =====

--- Books ---
1 - The Alchemist by Paulo Coelho - Available
2 - Atomic Habits by James Clear - Available
3 - Harry Potter by J.K. Rowling - Available

--- Members ---
101 - Rahul
102 - Aman

Searching for Atomic Habits:
2 - Atomic Habits by James Clear - Available

Borrowing book 1:
Book borrowed successfully.

--- Books ---
1 - The Alchemist by Paulo Coelho - Borrowed
2 - Atomic Habits by James Clear - Available
3 - Harry Potter by J.K. Rowling - Available

Returning book 1:
Book returned successfully.

--- Books ---
1 - The Alchemist by Paulo Coelho - Available
2 - Atomic Habits by James Clear - Available
3 - Harry Potter by J.K. Rowling - Available
fauzan:lib$ 2 2

```

7. What I Learned

Working on this Dart Library System assignment helped me understand how the different programming concepts we learned in class can be combined to create a complete working program. Before doing this assignment, I understood concepts like variables, functions, loops, classes, and inheritance individually, but I did not have a very clear idea of how they would work together in a real application. While creating the library system, I understood that each concept has a specific purpose and that combining them makes the program easier to organize and manage.

The first thing I understood better was the use of **classes and objects**. In this assignment, the library contains different types of information, such as books and members. Instead of keeping all this information in separate variables, I could create classes such as `LibraryBook` and `LibraryMember`. A class acts like a blueprint, while an object represents an actual item created from that blueprint. For example, instead of having separate variables for every book title and author, a `LibraryBook` object can contain all the information related to one book. This made the concept of classes much clearer to me because I could see how they are useful for representing real-world things in a program.

I also understood **constructors** better while working on the assignment. A constructor makes it possible to initialize an object when it is created. For example, when creating a book, its ID, title, and author can be provided immediately. This is much more organized than creating an empty object and assigning every value later. Earlier, constructors felt like just another syntax that had to be remembered, but using them in the library system helped me understand their actual purpose.

Another important concept I learned was **inheritance**. The assignment specifically required OOP with class inheritance, so I used `LibraryMember` as a parent class and `PremiumLibraryMember` as a child class. This helped me understand that inheritance allows one class to reuse properties and behavior from another class. A premium member is still a library member, but it can have additional functionality. The main thing that clicked for me here was that inheritance is useful when two types of objects have common characteristics but one of them needs some additional features. It also avoids unnecessarily repeating the same code in multiple classes.

The assignment also helped me understand **methods and functions** in a more practical way. Instead of putting the complete program inside one large `main()` function, different operations can be separated into functions or methods. For example, adding a book, displaying books, searching for a book, borrowing a book, and returning a book can each be handled separately. This makes the program easier to read and also makes it easier to find and fix problems. I realized that functions are not only used to reduce repeated code, but also help divide a bigger problem into smaller tasks.

I also used **loops** while working with the library data. Since a library can contain multiple books and members, it would not make sense to manually write code for every individual book. A loop can go through a collection and process each item. This helped me understand why loops are important when working with collections of data. The loop does the same type of operation repeatedly without requiring the same code to be written again and again.

One of the most useful parts of the assignment was working with **List, Map, and Set**. Before this assignment, I knew that they were different Dart collection types, but using them in one practical example made the differences easier to understand. A `List` can be used when we need to store multiple books or members. A `Map` is useful when we need to associate one piece of information with another, such as keeping track of which member has borrowed a particular book. A `Set` is useful when we only

want unique values, such as unique member names. Seeing these collections being used for different purposes helped me understand when each one should be selected.

The **borrowing and returning functionality** was also useful for understanding how program data changes while the program is running. When a book is borrowed, it should no longer be treated as available. When it is returned, its status should change again. This means that the program needs to update the relevant information whenever an operation is performed. I learned that it is important to think about how one operation affects other parts of the program instead of treating every function as completely separate.

While doing the assignment, I also faced some difficulties. The biggest difficulty was connecting all the concepts together. Understanding a class individually was not too difficult, but deciding which data should belong to which class and which operations should be handled by the library was more challenging. I had to think about the responsibilities of each class instead of putting everything in one place. This made me realize that writing a program is not only about knowing syntax; it is also about deciding how the program should be structured.

Another problem I faced was understanding **inheritance practically**. At first, I understood that one class could extend another class, but I was not completely sure why I would need it. After using `PremiumLibraryMember` with `LibraryMember`, the purpose became much clearer. The parent class can contain the common information and functionality, while the child class can add something specific to itself. This made inheritance feel more useful than just being a theoretical concept.

I also faced some issues while checking the borrowing and returning logic. It was necessary to make sure that the book status and the borrowed-book information were updated correctly. If a book was borrowed, it should appear as borrowed, and after returning it, it should become available again. Thinking through these cases helped me understand the importance of keeping related data consistent.

Another thing I learned was the importance of **testing the program** instead of assuming that the code is correct just because there are no obvious syntax errors. Running the program and checking its output helped me identify whether the functions were actually performing the expected operations. For example, I could check whether books were being added correctly, whether searching returned the correct book, and whether borrowing and returning changed the status correctly.

Overall, this assignment gave me a much better understanding of basic **Object-Oriented Programming in Dart**. The most important takeaway for me was that a program becomes easier to understand when it is divided into meaningful parts. Instead of thinking about the whole library system as one large problem, I could think about it as books, members, and library operations. Classes represent the different parts, functions and methods perform the operations, loops process multiple items, and collections store the data.

In conclusion, I learned that programming is not only about writing code that runs. It is also about organizing the code so that each part has a clear purpose. Through this assignment, I became more comfortable with classes, constructors, inheritance, functions, loops, and Dart collections such as `List`, `Map`, and `Set`. I also learned that debugging and testing are important parts of development. The main concept that clicked for me was how individual programming concepts can work together to model a real-world system in a structured way.

8. Conclusion

The Dart Library System demonstrates variables, functions, loops, classes, constructors, inheritance, List, Map, and Set in one console application. The assignment helped connect individual Dart concepts to a practical problem and improved my understanding of basic OOP design.